Phase 1: Forward-Looking Development

1.1 SCALABILITY:

THE GRID:

- Grid Size changed through a macro definition
- Coordinate system (uses a dynamic ascii-based numeral system)

DYNAMIC NUMERAL SYSTEM:

- Uses three main functionalities:
  - Setting the first and last characters of the system
  - Converting from custom numeral system to array indices (i,j)
  - Displaying the custom numerals with the correct spacing



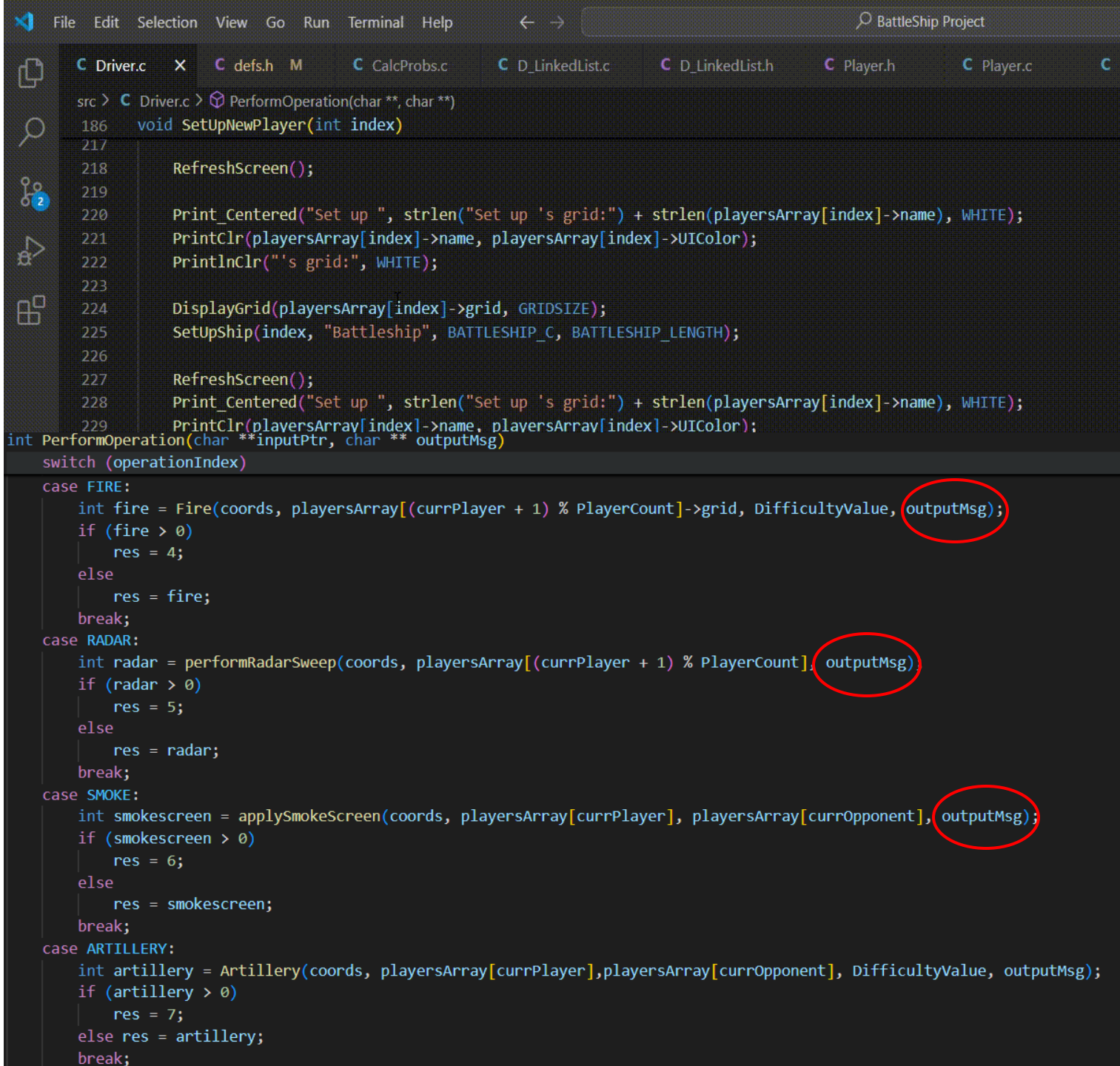
1.1 SCALABILITY:

MODULAR FUNCTIONALITY:

- Many functions developed in phase 1 were made to be reusable.

CENTRALIZATION OF FRONTEND CODE:

- All code responsible for printing, formatting, and drawing on the terminal is in the driver.
- Scaling up the game requires modifying a few macros and upgrading the driver, with no modification to backend code.
- Functions passed output messages back through each other to the driver which printed them.

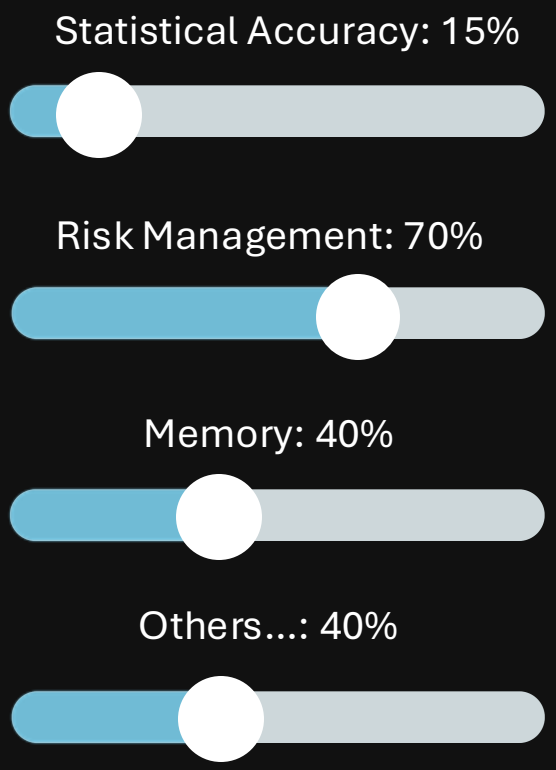


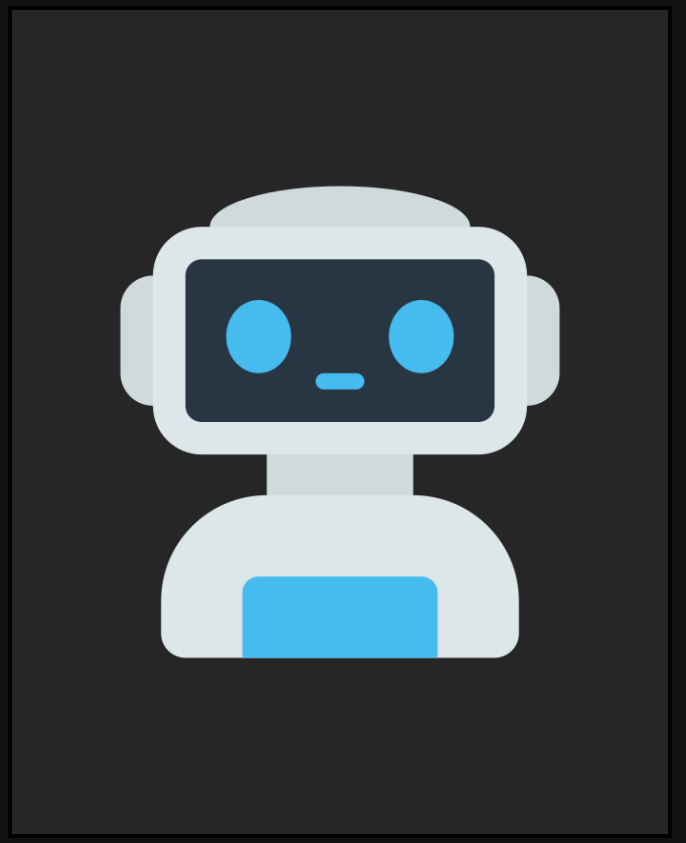
Phase 2: The Bot

1: MOTIVATION

GOAL:

- Achieve a bot whose intelligence can be fully customized through various variables:
  - Statistical Accuracy
  - Risk Management
  - Memory





2.1 PROBABILITY GRID

IDEA:

- Create a probability distribution out of every player's grid
- Stored in a separate 2D array data structure
- Modify the probability distribution according to the hits and misses that occur throughout the game

8	12	14	16	17	17	16	14	12	8
12	14	17	18	18	18	18	17	14	12
14	17	18	19	20	20	19	18	17	14
16	18	19	20	21	21	20	19	18	16
17	18	20	21	22	22	21	20	18	17
17	18	20	21	22	22	21	20	18	17
16	18	19	20	21	21	20	19	18	16
14	17	18	19	20	20	19	18	17	14
12	14	17	18	18	18	18	17	14	12
8	15	15	15	17	17	16	14	12	8

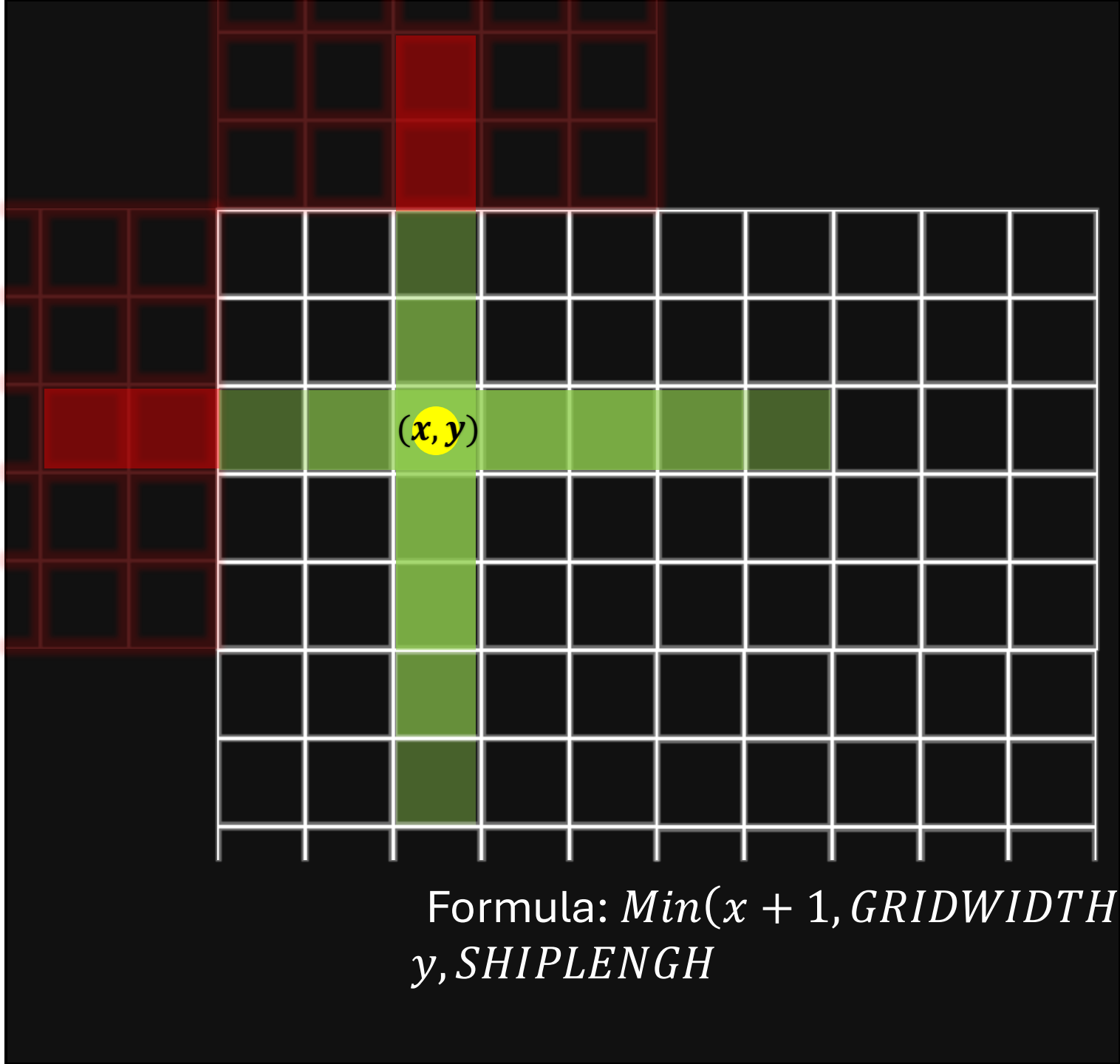
2.1.1 PROBABILITY CALCULATION

At α:

- For ship of length k, add k to the cell's probability
- Subtract out-of-bounds placements
- Do twice for both orientations
- Consider all the ships

ACTUAL IMPLEMENTATION:

- The above can be done using one formula rather than using loops.



2.1.2 UPDATING PROBABILITIES

IDEA:

- On a hit, add to each cell on the row and on the column of the hit the number of possible ways a ship might pass through it and pass through the hit at the same time.
- On a miss, deduct from each cell the number of ship placements that overlap with the miss.

SINKING SHIP:

- When a ship sinks, we recalculate the entire probability grid.

8	12	14	16 (+1)	17	17	16	14	12	8
12	14	17	18 (+2)	18	18	18	17	14	12
14 (+1)	17 (+2)	18 (+3)	X	20 (+4)	20 (+3)	19 (+2)	18 (+1)	17	14
16	18	19	20 (+3)	21	21	20	19	18	16
17	18	20	21 (+2)	22	22	21	20	18	17
17	18	20	21 (+1)	22	22	21	20	18	17
16	18	19	20	21	21	20	19	18	16
14	17	18	19	20	20	19	18	17	14
12	14	17	18	18	18	18	17	14	12
8	15	15	15	17	17	16	14	12	8

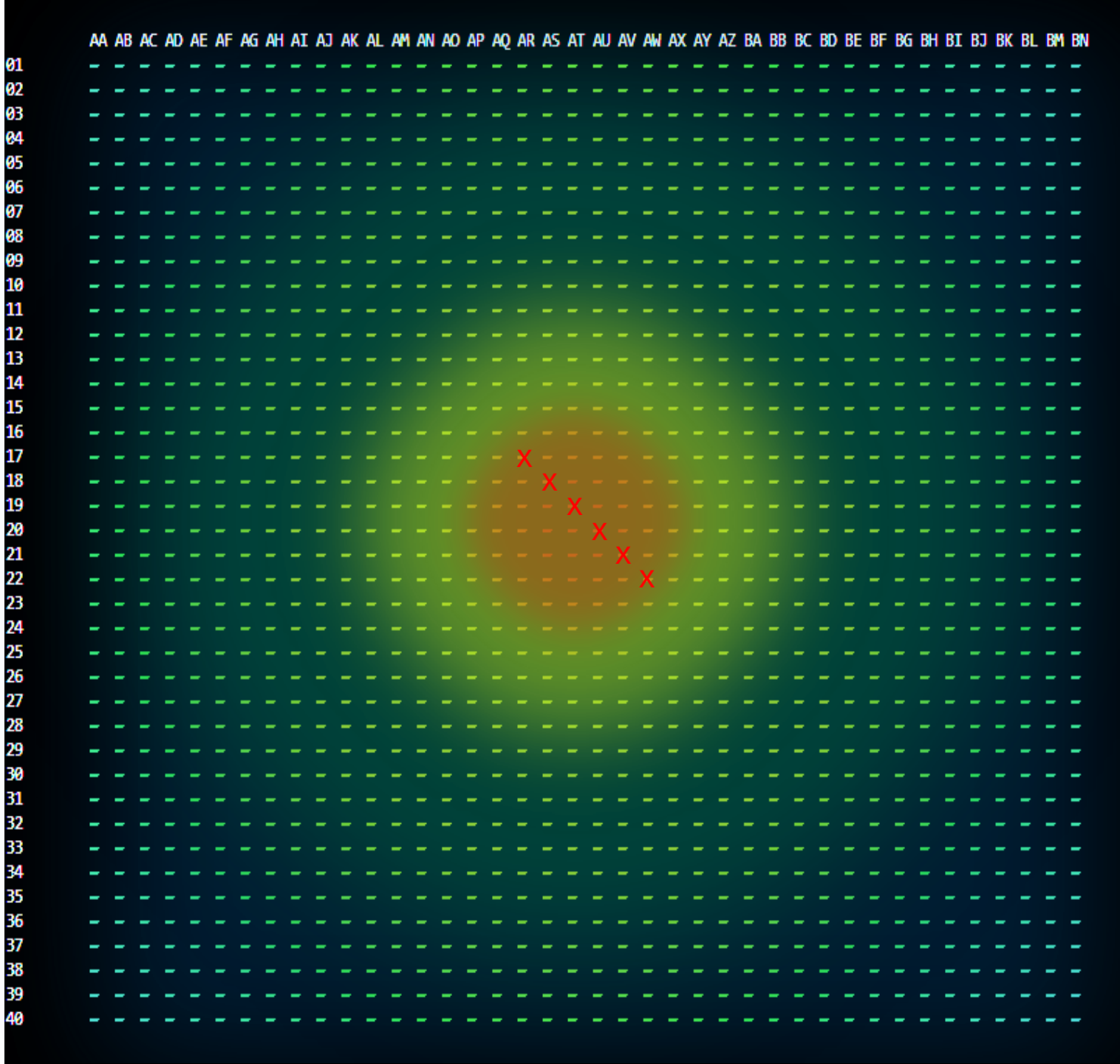
2.2: PREDICTABILITY

PROBLEM:

- Using the normal highest probability targeting method deems the bot extremely predictable
- Using a heatmap also leads to very predictable behavior

OUR GOALS:

- Randomize, but along certain lines. (not too much randomization)
- Bot favors higher probability, but not too often.
- Keep track of the best next cells at all time



2.2.1: REGIONALIZING THE GRID

- Solution to the semi-randomization problem.
- Have full control over the resolution of the statistical model.
- Divide the grid into n^2/k^2 $k \times k$ regions
- Target according to probability regions rather than individual cell probability
- Order regions according to the average probability of each

NEW PROBLEM:

- Reordering the regions takes $O(n^2/k^2) = O(n^2)$ time.
- Performed after every turn.
- $O(n^2)$ every turn is **too much**.

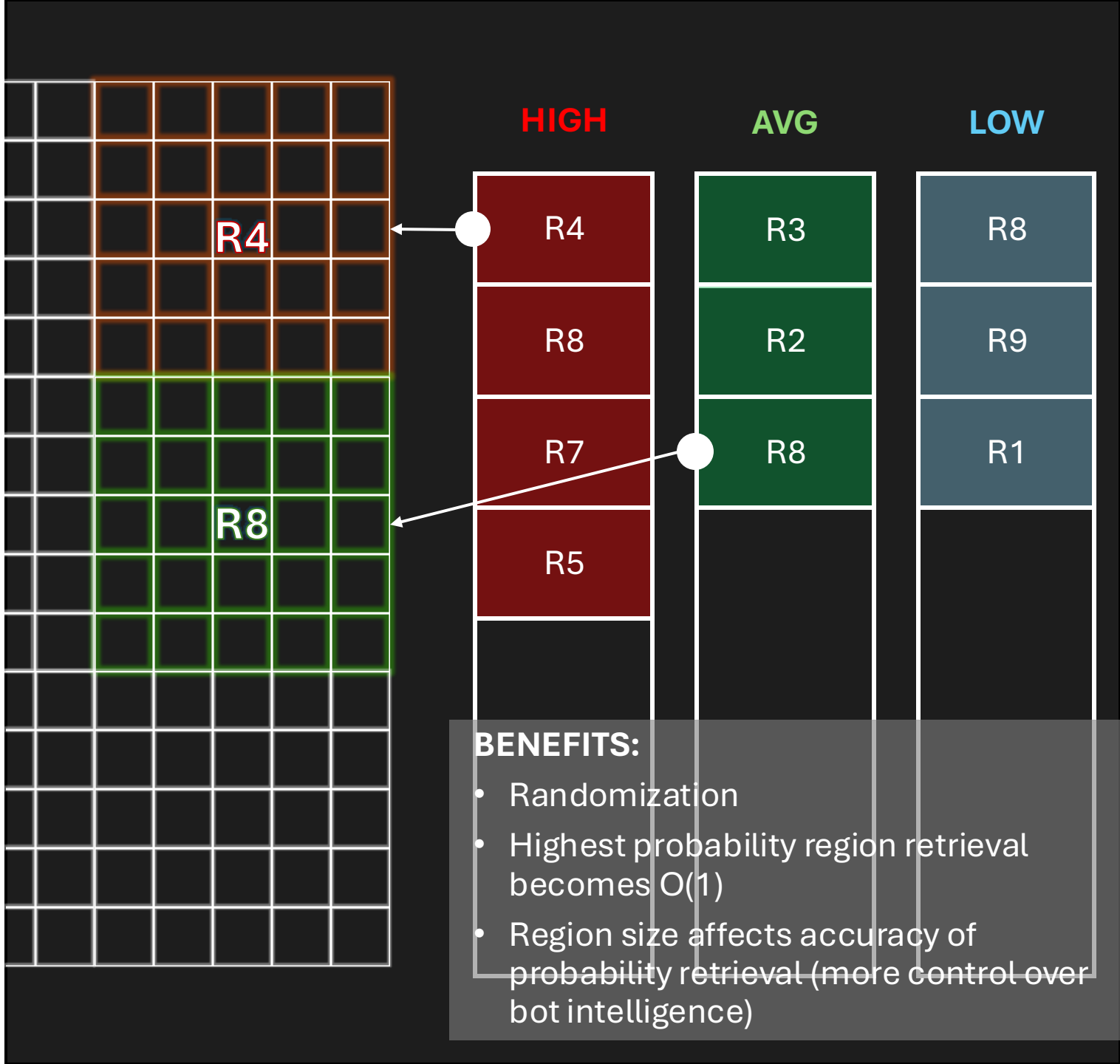


The smaller the regions the more accurate the targeting is.

2.2.1: REGIONALIZING THE GRID

SOLUTION:

- Split the regions into 3 probability categories:
  - High,
  - Average
  - Low
- No need to sort the regions inside the categories
- For highest probability, access the high category, and so on…
- After region changes, access the region and move it to the correct category.
- O(n/k) to find region (still more efficient than sorting an n/k list)
- But we will do better than O(n/k), we will achieve O(1)…



BENEFITS:

- Randomization
- Highest probability region retrieval becomes O(1)
- Region size affects accuracy of probability retrieval (more control over bot intelligence)

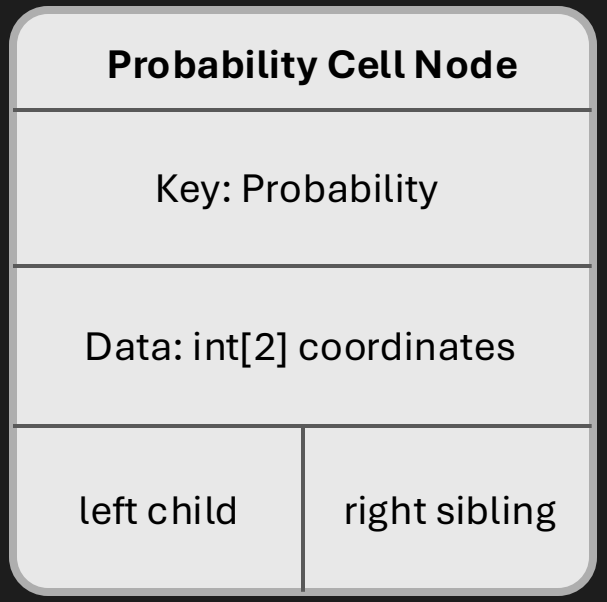
2.2.2: ORDERING THE REGION'S CELLS

IDEA:

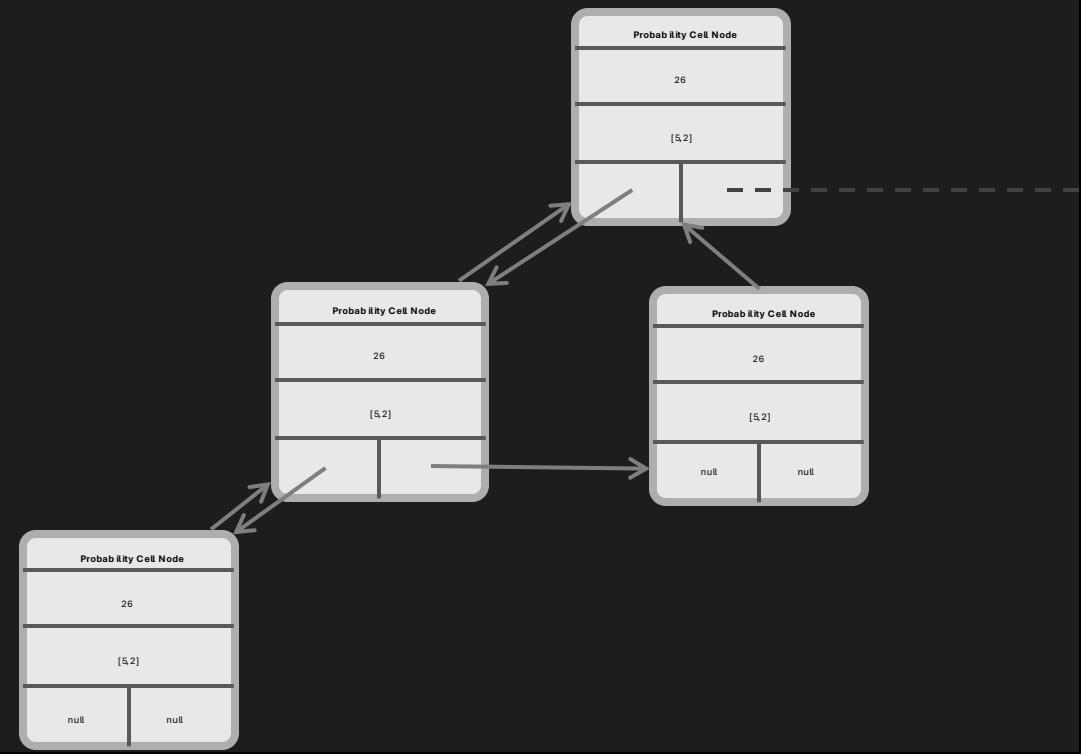
- Order the cells of a region using a priority queue
- Keeps track of the highest probability of the region
- O(1) retrieval
- O(k) for updating the region's probability in case of hit or miss (k = region size = constant)
- Used binomial heaps just for fun

PROBLEM:

- How to efficiently access the heaps to update them?
- Searching in O(n/k) defeats the whole purpose of our methods



Binomial Heap of A Probability Region



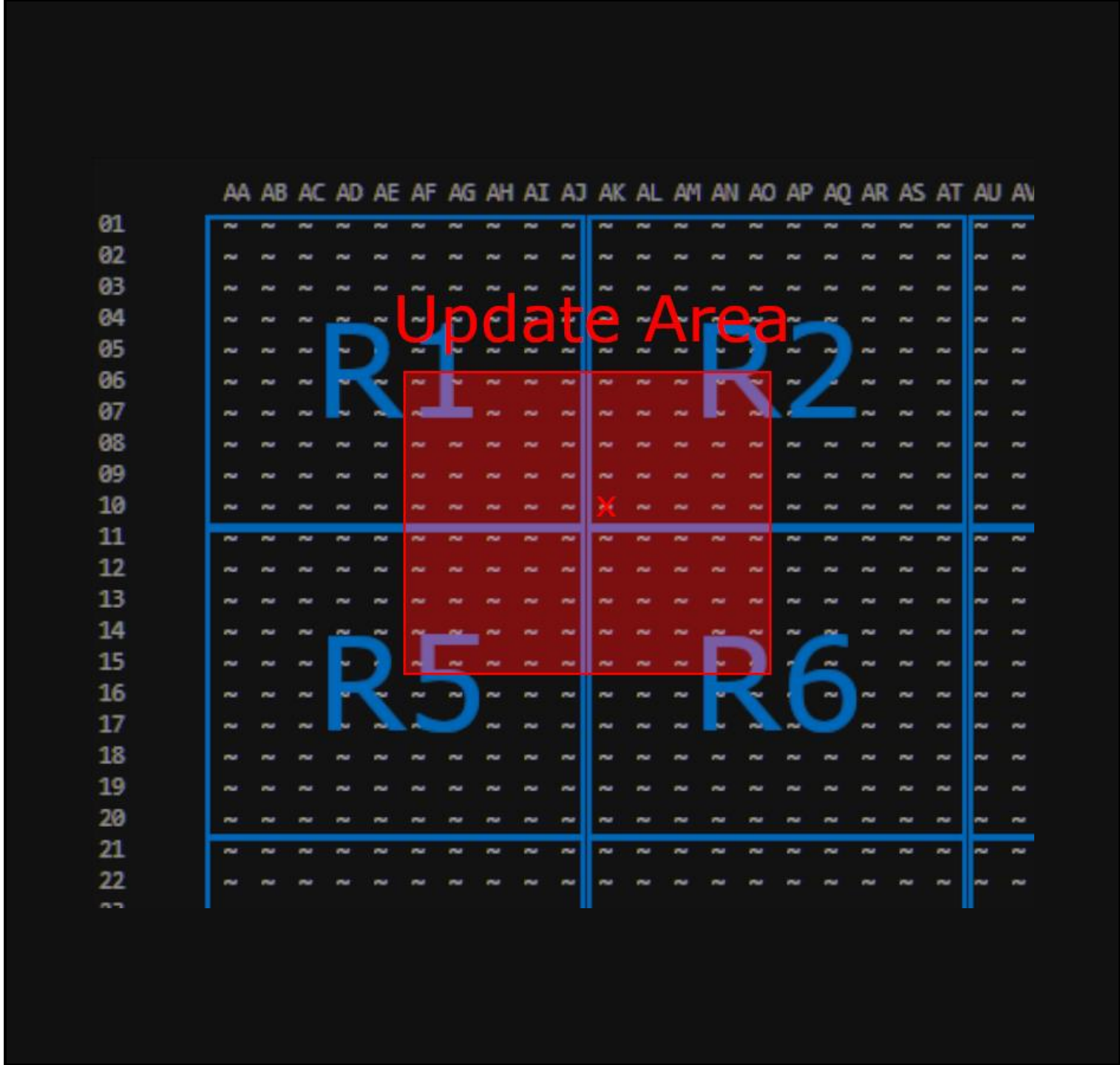
2.2.3: UPDATING THE REGIONS

IDEA:

- After a hit or miss, the area surrounding the cell must be updated.
- Area width and height are both double the longest ship
- For small enough region size, area might overlap multiple regions

PROBLEM ON HOLD:

- How to efficiently access each region to update it?



2.2.4: HASHSET OF REGIONS

IDEA:

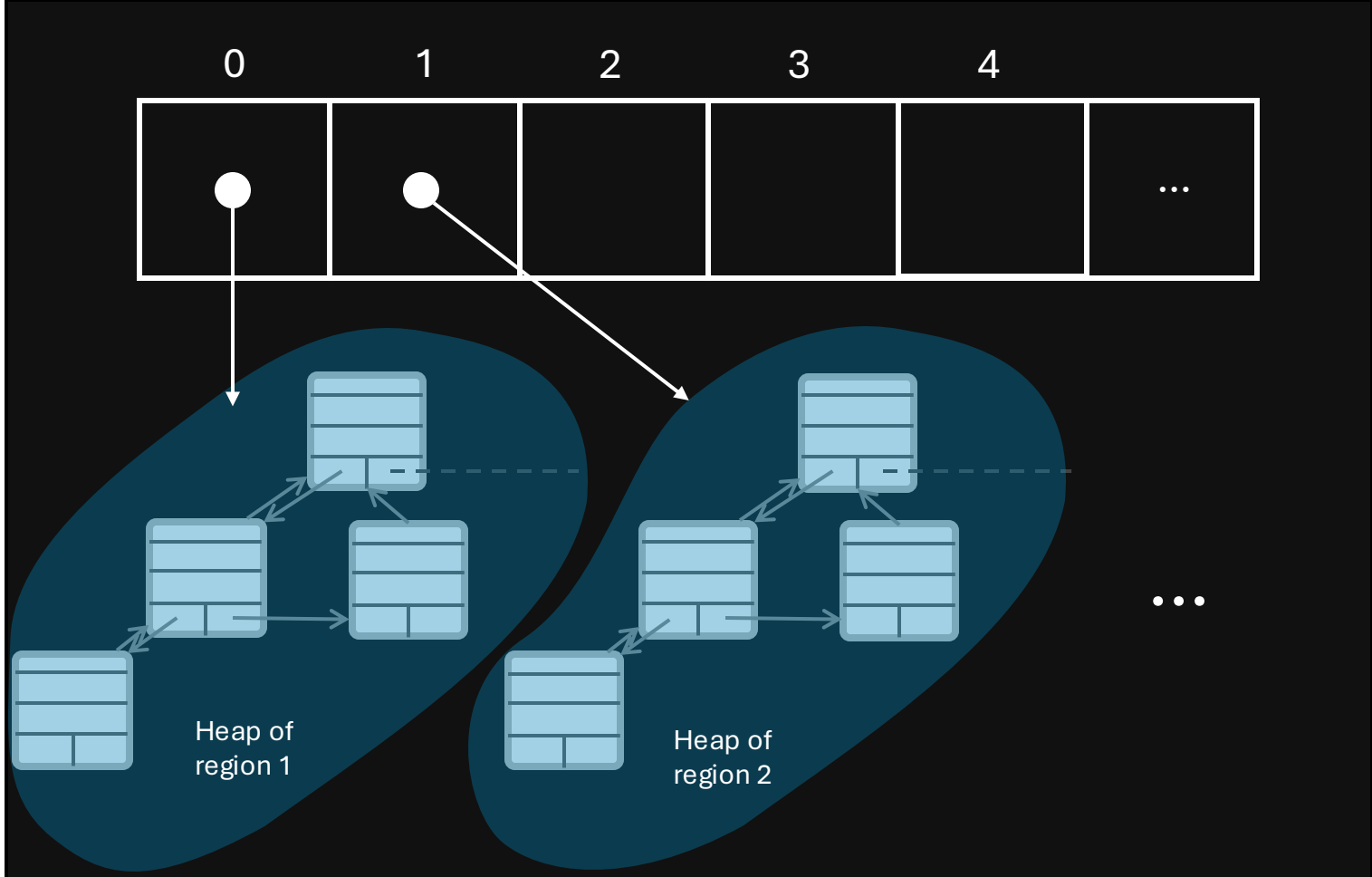
- Hash every region heap to a unique index in an array
- Number the regions from 0 up to n/k – 1
- Any (i, j) coordinate inside region R_l should hash to index $l - 1$. $(1 \leq l \leq n/k)$
- Use a special hashing function called RegionHash().

PROBLEM SOLVED:

- ~~How to efficiently access each region to update it?~~

PROBLEM LEFT:

- Update region category faster than O(n/k).

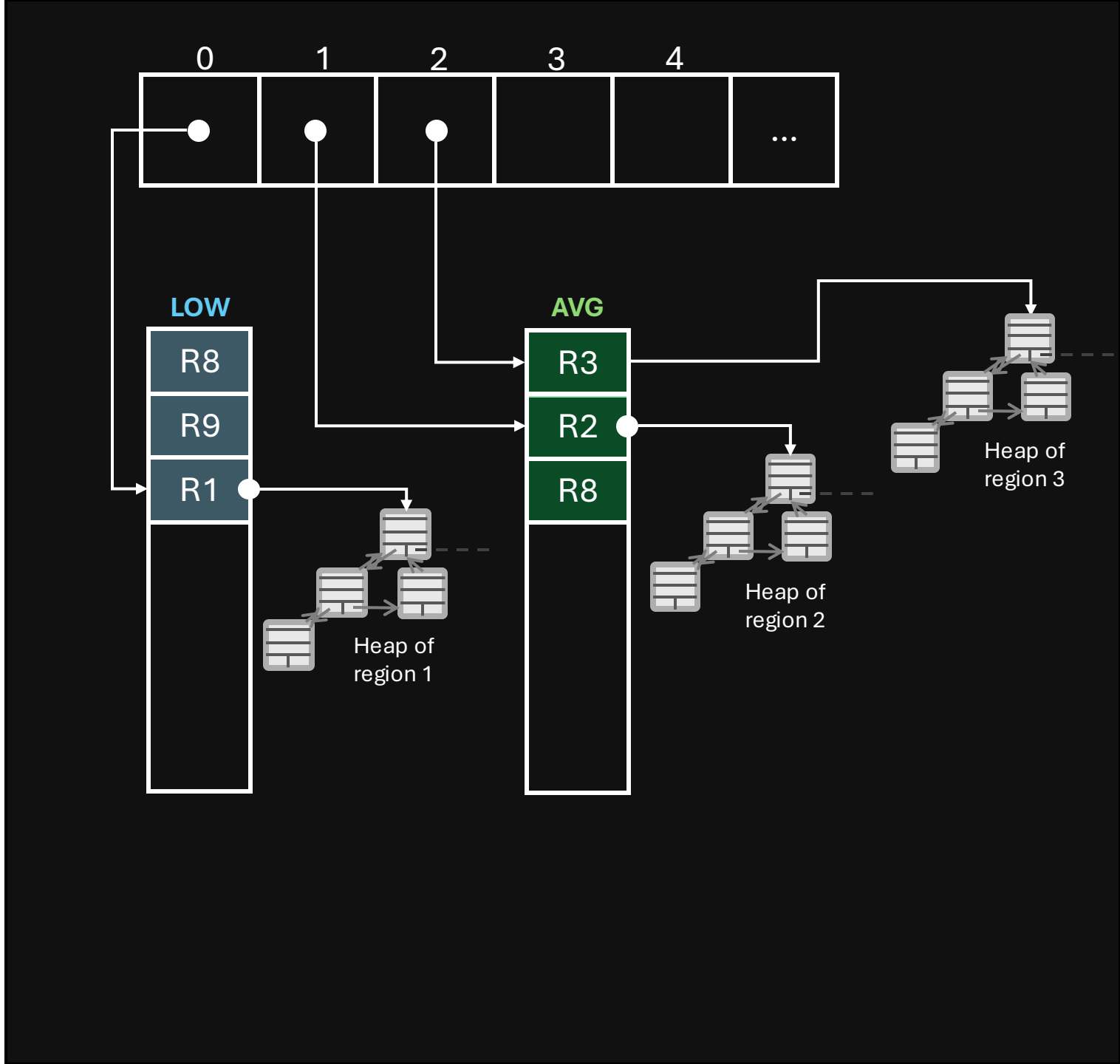


Need to show the region hash formula

2.2.5: FINAL REGION SYSTEM

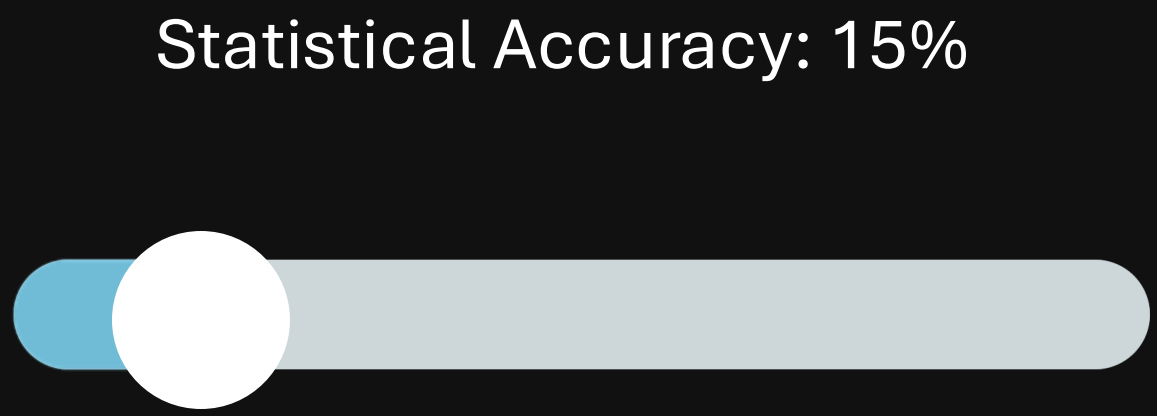
OVERVIEW:

- After probability grid is calculated, region heaps are initialized and inserted into the correct category list.
- The slot in the hash table is a pointer that points to that node in the linked list.
- We have O(1) access to the region in the list.
- Therefore, moving the region to the right category can now be performed in O(1).
- Bot picks next best cell by pulling the head of a binomial heap in one of the three categories
- Bot updates regions surrounding target by directly accessing the HashSet of heaps.



2.2.6: HIDDEN BENEFITS

- Our regionalization of the graph acts like a resolution screen that controls the bot's statistical accuracy.
- This resolution can be raised and lowered, offering significant control over how smart our bot is.
- More control over how smart the bot can be follows in the upcoming section…



2.3: RISK

IDEA:

- Opponent's actions affect bot's behavior
- Bot recognizes three risk levels:
  - Low risk
  - Normal risk
  - High risk
- Risk is determined by calculating:
  - How accurate the opponent has been
  - Rate of success of opponent (for predicting future hits)
  - Number of ships sunk by bot compared to opponent

$$\text{Accuracy} = \frac{Opponent's\ Hits}{Opponent's\ total\ shots}$$

$$\text{Success Rate} = \frac{Opponent's\ Hits\ in\ last\ \boldsymbol{t}\ turns}{t}$$

$$\text{Sunk Ratio} = \frac{Opponent's\ Sunk\ Ships}{Bot's\ Sunk\ Ships}$$

2.3.1: RISK DICTATES THE GAME

IDEA:

- Bot starts at normal risk, performing its attack routine
- At low risk, the bot takes more risks:
  - targets lower probabilities more often
  - higher chance of attacking somewhere else while sinking a ship
- At normal or high risk, the bot takes fewer risks and tries to lower the risk:
  - Focuses its attacks to highest probability regions, and often searches for hottest cells (O(n/k))
  - depending on how many ships it can see it chooses whether to focus more on sinking the ships or searching for more ships

RISK CONTROL

IDEA:

- By changing the risk values (high, avg, low) that affect the bot's behavior, we can control how dynamically the bot reacts to the opponent's success.
- This is yet another dimension we can work with to control the bot's intelligence level.

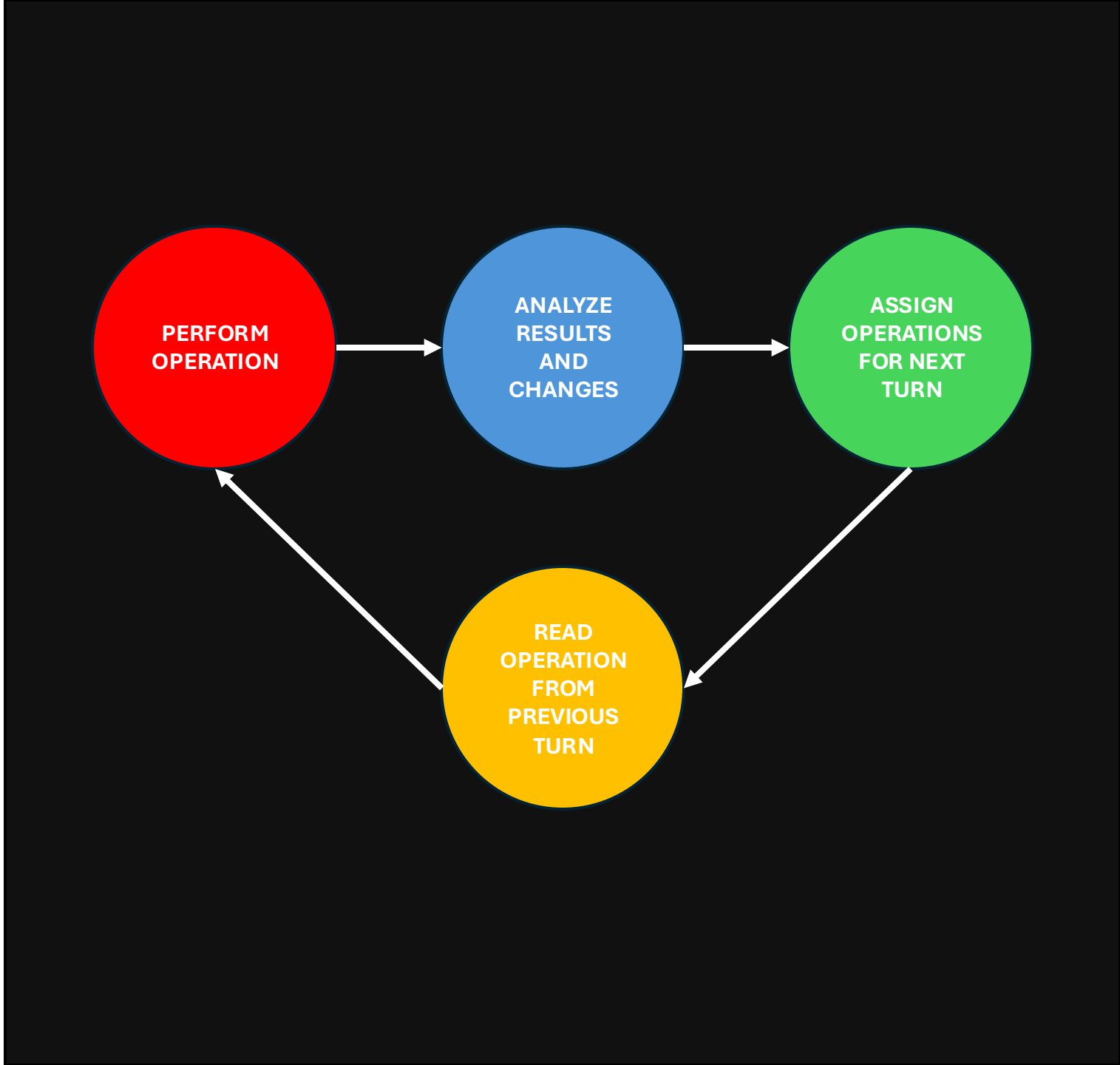
2.4: THE BRAIN

IDEA:

- The bot's decisions must be memorized across turns
- For example:
  - Hot cells after a ship was hit
  - previous decisions for adapting new techniques

PROBLEM:

- How to store the decisions and access them?



2.4.1: THE STACK

IDEA:

- Use a stack that keeps track of the bot's tasks
- Use doubly linked list for inserting high priority tasks on top and low from bottom
- Store a task node containing function pointers and their respective argument arrays
- Allows us to implement different tactics in later versions of the game and easily connect them to the bot's brain:
  - New tactic = New function
  - Functions assign tasks with different priorities to the stack depending on their logic
  - Any advancement to the bot's brain would only require adjustments in the stack logic

